

EMSI — Développement Front-End

TP Vue.js — Mini Todo App

Code complet fourni + comparaisons React ↔ Vue

Projet	TodoVue — une todo app complète en Vue.js 3
Objectif	Comprendre Vue.js en comparant chaque concept avec React
Stack	Vue 3 (Composition API) + TypeScript + Vite + json-server

Partie 1 : Créer le projet

```
npm create vue@latest todo-vue

# TypeScript ? Yes | Vue Router ? Yes | Pinia ? Yes

cd todo-vue && npm install && npm run dev
```

Dans un 2ème terminal :

```
npx json-server --watch db.json --port 4000
```

Copiez votre db.json de TaskFlow React.

Comparaison des structures :

Fichier principal	
React src/App.tsx (un seul fichier JS)	Vue 3 src/App.vue (3 sections : script + template + style)
Pages	
React src/pages/ (convention libre)	Vue 3 src/views/ (convention Vue)
Point d'entrée	
React src/main.tsx	Vue 3 src/main.ts

➤ Q1 : Un fichier .vue a 3 sections séparées. Un fichier .tsx mélange tout. Quel avantage/inconvénient de chaque approche ?

Partie 2 : TodoList complet

Avant d'écrire le code Vue, voici les correspondances avec React :

State réactif	
React const [tasks, setTasks] = useState([])	Vue 3 const tasks = ref([])
State string	
React const [title, setTitle] = useState('')	Vue 3 const title = ref('')
Fetch au montage	
React useEffect(() => { fetch()... }, [])	Vue 3 onMounted(async () => { fetch()... })
Ajouter au state	
React setTasks(prev => [...prev, data])	Vue 3 tasks.value.push(data)
Supprimer du state	
React setTasks(prev => prev.filter(t => t.id !== id))	Vue 3 tasks.value = tasks.value.filter(...)
Two-way binding (3 lignes vs 1)	
React value={title} onChange={e => setTitle(e.target.value)}	Vue 3 v-model="title"
Boucle	
React {tasks.map(t => <li key={t.id}>)}	Vue 3 <li v-for="t in tasks" :key="t.id">
Conditionnel	
React {loading && Chargement }	Vue 3 <p v-if="loading">Chargement</p>
Événement	
React onClick={handleClick}	Vue 3 @click="handleClick"

Remplacez TOUT le contenu de src/views/HomeView.vue :

```
<script setup lang="ts">
import { ref, onMounted } from 'vue';

interface Task { id: string; title: string; done: boolean; }

const tasks = ref<Task[]>([]);
const newTitle = ref('');
const loading = ref(true);

onMounted(async () => {
  const res = await fetch('http://localhost:4000/tasks');
  tasks.value = await res.json();
  loading.value = false;
});
```

```

async function addTask() {
  if (!newTitle.value.trim()) return;
  const res = await fetch('http://localhost:4000/tasks', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ title: newTitle.value, done: false }),
  });
  tasks.value.push(await res.json());
  newTitle.value = '';
}

async function deleteTask(id: string) {
  await fetch('http://localhost:4000/tasks/' + id, { method: 'DELETE' });
  tasks.value = tasks.value.filter(t => t.id !== id);
}

async function toggleTask(task: Task) {
  const res = await fetch('http://localhost:4000/tasks/' + task.id, {
    method: 'PUT',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ ...task, done: !task.done }),
  });
  const data = await res.json();
  const i = tasks.value.findIndex(t => t.id === data.id);
  tasks.value[i] = data;
}
</script>

<template>
  <div style="padding: 2rem; max-width: 600px; margin: 0 auto;">
    <h1 style="color: #42b883;">Mes Tâches ({{ tasks.length }})</h1>
    <div style="display: flex; gap: 8px; margin: 16px 0;">
      <input v-model="newTitle" placeholder="Nouvelle tâche..."
        @keyup.enter="addTask"
        style="flex: 1; padding: 10px; border-radius: 6px; border: 1px solid #ddd;" />
      <button @click="addTask"
        style="padding: 10px 20px; background: #42b883; color: white; border: none;
border-radius: 6px;">
        + Ajouter
      </button>
    </div>
    <p v-if="loading" style="color: #999;">Chargement...</p>
    <p v-else-if="tasks.length === 0" style="color: #999;">Aucune tâche.</p>
    <ul v-else style="list-style: none; padding: 0;">
      <li v-for="task in tasks" :key="task.id"
        style="display: flex; align-items: center; gap: 12px; padding: 14px; margin-bottom:
8px; background: white; border-radius: 8px;">
        <input type="checkbox" :checked="task.done" @change="toggleTask(task)" />
        <span :style="{ flex: 1, textDecoratoin: task.done ? 'line-through' : 'none' }">
          {{ task.title }}
        </span>
        <button @click="deleteTask(task.id)" style="background:none; border:none;
cursor:pointer;">Del</button>
      </li>
    </ul>
  </div>
</template>

```

- Q2 : En React, l'input nécessite `useState` + `value` + `onChange` (3 éléments). En Vue, v-model fait tout en 1. Pourquoi React ne propose-t-il pas de two-way binding natif ?
- Q3 : En React, `tasks.push(data)` est INTERDIT (immutabilité). En Vue, `tasks.value.push(data)` fonctionne. Expliquez pourquoi avec le mot « Proxy ».
- Q4 : React utilise `useEffect(fn, [])` pour le montage. Vue utilise `onMounted(fn)`. Lequel est plus lisible ? Pourquoi React a-t-il choisi un tableau de dépendances ?

Partie 3 : Composant TodoItem

Communication parent ↔ enfant :

Props (parent → enfant)	
React props: { task: Task }	Vue 3 defineProps<{ task: Task }>()
Action (enfant → parent)	
React onDelete={deleteTask} // callback prop	Vue 3 @delete="deleteTask" // event listener
Côté enfant	
React // L'enfant appelle : props.onDelete(id)	Vue 3 // L'enfant appelle : emit('delete', id)

Créez src/components/TodoItem.vue :

```
<script setup lang="ts">
interface Task { id: string; title: string; done: boolean; }

defineProps<{ task: Task }>();
const emit = defineEmits<{
  toggle: [task: Task];
  delete: [id: string];
}>();
</script>

<template>
  <li
    style="display: flex; align-items: center; gap: 12px; padding: 14px; margin-bottom: 8px;
background: white; border-radius: 8px;"
  >
    <input type="checkbox" :checked="task.done" @change="emit('toggle', task)" />
    <span :style="{
      flex: 1,
      textDecoration: task.done ? 'line-through' : 'none',
      color: task.done ? '#999' : '#333'
    }">
      {{ task.title }}
    </span>
    <RouterLink :to="'/tasks/' + task.id" style="color: #42b883;">Détails</RouterLink>
    <button @click="emit('delete', task.id)" style="background:none; border:none;
cursor:pointer;">Del</button>
  </li>
</template>
```

Dans `HomeView.vue`, importez et utilisez :

```
// Dans <script setup> :  
import TodoItem from '@components/ToDoItem.vue';  
  
// Dans <template>, remplacez le <li v-for> par :  
<TodoItem v-for="task in tasks" :key="task.id"  
  :task="task" @toggle="toggleTask" @delete="deleteTask" />
```

- Q5 : React passe des **FONCTIONS** en props (`onDelete={fn}`). Vue émet des **ÉVÉNEMENTS** (`@delete`). Quelle approche est plus proche du HTML natif ? Pourquoi ?
- Q6 : En React, si on oublie de passer `onDelete` en prop, l'app crash au clic. En Vue, si on oublie `@delete`, que se passe-t-il ? (indice : l'événement est silencieusement ignoré)

Partie 4 : Vue Router

Correspondances routing :

Définir une route	
React <code><Route path='/tasks/:id' element={<Detail/>} /></code>	Vue 3 <code>{ path: '/tasks/:id', component: Detail }</code>
Lire les params	
React <code>const { id } = useParams()</code>	Vue 3 <code>useRoute().params.id</code>
Navigation programmatique	
React <code>const navigate = useNavigate()</code>	Vue 3 <code>const router = useRouter()</code>
Naviguer	
React <code>navigate('/tasks')</code>	Vue 3 <code>router.push('/tasks')</code>
Lien	
React <code><Link to='/tasks'>Liste</Link></code>	Vue 3 <code><RouterLink to='/tasks'>Liste</RouterLink></code>
Afficher la page enfant	
React <code><Outlet /></code>	Vue 3 <code><RouterView /></code>

Modifiez src/router/index.ts :

```
import { createRouter, createWebHistory } from 'vue-router';
import HomeView from '../views/HomeView.vue';

const router = createRouter({
  history: createWebHistory(import.meta.env.BASE_URL),
  routes: [
    { path: '/', name: 'home', component: HomeView },
    { path: '/about', name: 'about', component: () => import('../views/AboutView.vue') },
    { path: '/tasks/:id', name: 'task', component: () => import('../views/TaskView.vue') },
  ],
});
export default router;
```

Créez src/views/TaskView.vue :

```
<script setup lang="ts">
import { ref, onMounted } from 'vue';
import { useRoute, useRouter } from 'vue-router';

interface Task { id: string; title: string; done: boolean; }
const route = useRoute();
const router = useRouter();
const task = ref<Task | null>(null);

onMounted(async () => {
  try {
    const res = await fetch('http://localhost:4000/tasks/' + route.params.id);
    if (!res.ok) { router.push('/'); return; }
    task.value = await res.json();
  }
});
</script>
```

```

    } catch { router.push('/') }
  });
</script>

<template>
  <div v-if="task" style="padding: 2rem; max-width: 600px; margin: 0 auto;">
    <div style="background: white; padding: 24px; border-radius: 12px;">
      <h1 style="color: #42b883;">{{ task.title }}</h1>
      <p>Statut : {{ task.done ? 'Terminée' : 'En cours' }}</p>
      <RouterLink to="/" style="color: #42b883; margin-top: 20px; display:
inline-block;">Retour</RouterLink>
    </div>
  </div>
  <p v-else style="padding: 2rem; color: #999;">Chargement...</p>
</template>

```

➤ Q7 : React Router utilise `useParams()` + `useNavigate()` (2 hooks). Vue Router utilise `useRoute()` + `useRouter()` (2 hooks aussi). La logique est-elle vraiment différente ou juste les noms ?

➤ Q8 : En React, les routes sont définies DANS le JSX (`<Routes><Route .../></Routes>`). En Vue, elles sont dans un FICHIER de config (`router/index.ts`). Quel avantage à la séparation de Vue ?

Partie 5 : Pinia — State management

Correspondances state management :

Créer un store	
React createSlice({ name, initialState, reducers })	Vue 3 defineStore('name', () => { ... })
Appeler une action	
React dispatch(addTask(title))	Vue 3 store.addTask(title)
Lire le state	
React useSelector(state => state.tasks.items)	Vue 3 store.tasks
Muter le state	
React state.loading = true // via Immer	Vue 3 loading.value = true // directement
Nombre de fichiers	
React 3 fichiers : slice + store + Provider	Vue 3 1 fichier : defineStore()

Créez src/stores/tasks.ts :

```
import { defineStore } from 'pinia';
import { ref } from 'vue';

interface Task { id: string; title: string; done: boolean; }

export const useTaskStore = defineStore('tasks', () => {
  const tasks = ref<Task[]>([]);
  const loading = ref(true);
  const error = ref<string | null>(null);

  async function fetchTasks() {
    try {
      const res = await fetch('http://localhost:4000/tasks');
      tasks.value = await res.json();
    } catch { error.value = 'Erreur chargement'; }
    finally { loading.value = false; }
  }

  async function addTask(title: string) {
    const res = await fetch('http://localhost:4000/tasks', {
      method: 'POST', headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ title, done: false }),
    });
    tasks.value.push(await res.json());
  }

  async function deleteTask(id: string) {
    await fetch('http://localhost:4000/tasks/' + id, { method: 'DELETE' });
    tasks.value = tasks.value.filter(t => t.id !== id);
  }

  async function toggleTask(task: Task) {
```

```

    const res = await fetch('http://localhost:4000/tasks/' + task.id, {
      method: 'PUT', headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ ...task, done: !task.done }),
    });
    const data = await res.json();
    const i = tasks.value.findIndex(t => t.id === data.id);
    tasks.value[i] = data;
  }

  return { tasks, loading, error, fetchTasks, addTask, deleteTask, toggleTask };
});

```

Simplifiez HomeView.vue :

```

<script setup lang="ts">
import { ref, onMounted } from 'vue';
import { useTaskStore } from '@stores/tasks';
import TodoItem from '@components/TodoItem.vue';

const store = useTaskStore();
const newTitle = ref('');

onMounted(() => store.fetchTasks());

function handleAdd() {
  if (!newTitle.value.trim()) return;
  store.addTask(newTitle.value);
  newTitle.value = '';
}
</script>

<template>
  <div style="padding: 2rem; max-width: 600px; margin: 0 auto;">
    <h1 style="color: #42b883;">Mes Tâches ({{ store.tasks.length }})</h1>

    <div style="display: flex; gap: 8px; margin: 16px 0;">
      <input v-model="newTitle" placeholder="Nouvelle tâche..."
        @keyup.enter="handleAdd"
        style="flex: 1; padding: 10px; border-radius: 6px; border: 1px solid #ddd;" />
      <button @click="handleAdd"
        style="padding: 10px 20px; background: #42b883; color: white; border: none;
border-radius: 6px;">
        + Ajouter
      </button>
    </div>
  </div>

```

```

<p v-if="store.loading" style="color: #999;">Chargement...</p>
<p v-else-if="store.tasks.length === 0" style="color: #999;">Aucune tâche.</p>

<ul v-else style="list-style: none; padding: 0;">
  <TodoItem
    v-for="task in store.tasks"
    :key="task.id"
    :task="task"
    @toggle="store.toggleTask"
    @delete="store.deleteTask"
  />
</ul>
</div>
</template>

```

Modifiez src/App.vue :

```

<script setup>
import { RouterLink, RouterView } from 'vue-router'
</script>

<template>
  <header style="background: #42b883; color: white; padding: 1rem 2rem; display: flex;
justify-content: space-between; align-items: center;">
    <h2 style="margin: 0;">TodoVue</h2>
    <nav style="display: flex; gap: 16px;">
      <RouterLink to="/" style="color: white; text-decoration: none;">Tâches</RouterLink>
      <RouterLink to="/about" style="color: white; text-decoration:
none;">About</RouterLink>
    </nav>
  </header>
  <RouterView />
</template>

```

➤ Q9 : Redux Toolkit nécessite : `createSlice` + `configureStore` + `Provider` + `useSelector` + `useDispatch`. Pinia nécessite : `defineStore` + `useTaskStore()`. Comptez les concepts à apprendre pour chacun.

➤ Q10 : En Redux, on écrit `dispatch(addTask(title))`. En Pinia, on écrit `store.addTask(title)`. Lequel est plus intuitif ? Redux a-t-il un avantage que Pinia n'a pas ?

Redux a les DevTools avec time-travel debugging (revenir en arrière dans le temps). Pinia a aussi des DevTools mais moins puissants.

Partie 6 : Comparaison finale

Remplissez ce tableau avec ce que vous avez appris dans ce TP :

Concept	React	Vue 3
State local	<code>const [x, setX] = useState(0)</code>	
Two-way binding	<code>value={x} onChange={e => set(e.target.value)}</code>	
Fetch au montage	<code>useEffect(() => { fetch(...) }, [])</code>	
Ajouter au state	<code>setItems(prev => [...prev, data])</code>	
Boucle template	<code>items.map(i => <li key={i.id}>...)</code>	
Conditionnel	<code>{ condition && <div>...</div> }</code>	
Event	<code>onClick={handler}</code>	
Props enfant	<code><Child name={x} /></code>	
Callback parent	<code><Child onDelete={handler} /></code>	
Router lien	<code><Link to='/x'>Lien</Link></code>	
Params URL	<code>const { id } = useParams()</code>	
Navigate	<code>const nav = useNavigate(); nav('/')</code>	
State global	<code>createSlice + useSelector + dispatch</code>	
Appeler action	<code>dispatch(addTask(title))</code>	
Style isolé	CSS Modules / <code>.module.css</code>	

- Q11 : Quels concepts sont IDENTIQUES entre React et Vue (juste le nom qui change) ? Quels concepts sont FONDAMENTALEMENT différents ?
- Q12 : Vue a v-model (two-way binding natif) et les mutations directes (push, splice). React exige l'immuabilité et onChange explicite. Quel framework est plus « magique » ? Est-ce un avantage ou un inconvénient ?
- Q13 : Vous devez créer une app e-commerce avec 50+ pages, une équipe de 10 développeurs, et un dashboard admin complexe. React ou Vue ? Justifiez.
- Q14 : Un débutant qui n'a jamais fait de JavaScript framework veut apprendre. Vous lui conseillez React ou Vue en premier ? Pourquoi ?

Récap : structure finale

```
todo-vue/  
  src/  
    ├── App.vue           ← Layout + RouterView  
    ├── main.ts           ← createApp + router + pinia  
    ├── views/  
    │   ├── HomeView.vue  ← useTaskStore + TodoItem  
    │   ├── TaskView.vue  ← useRoute + onMounted  
    │   └── AboutView.vue  
    ├── components/  
    │   └── TodoItem.vue   ← defineProps + defineEmits  
    ├── stores/  
    │   └── tasks.ts       ← Pinia store CRUD  
    └── router/  
        └── index.ts       ← 3 routes
```